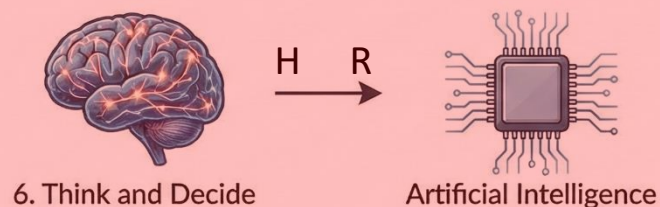
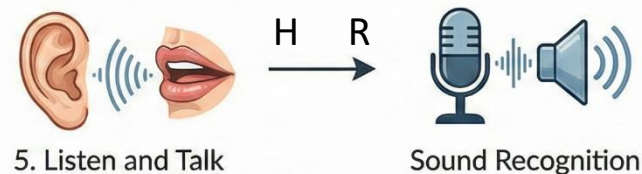
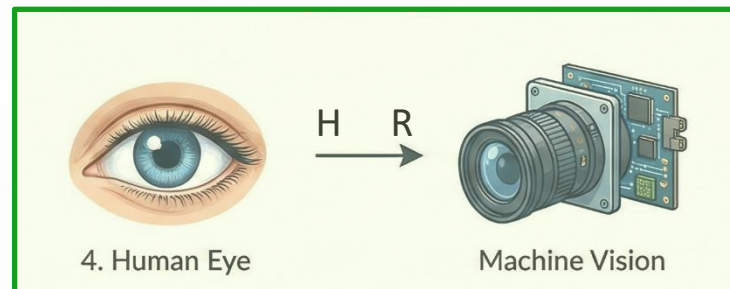
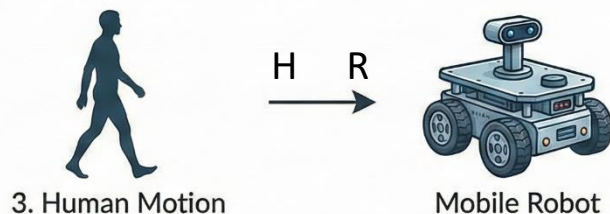
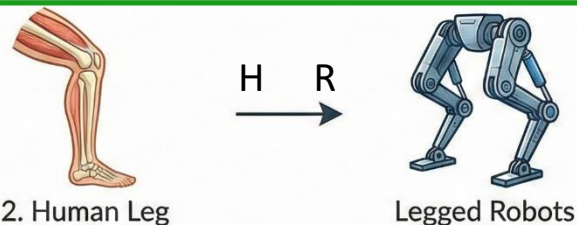
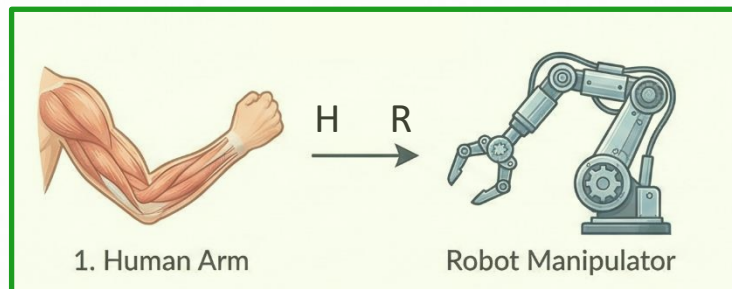


# ARAS Robotics & Computer Vision

## Chapter 5: Neural network foundations (Review)

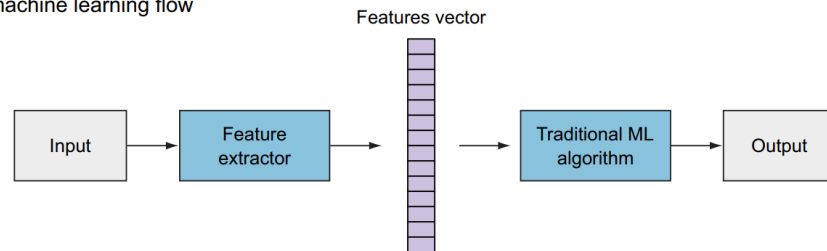
perceptron; MLP; activations; loss functions; backpropagation; optimization; regularization.



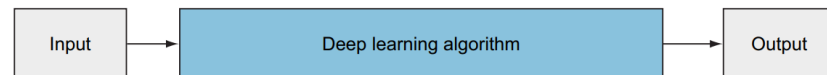
Neural networks learn complex patterns through layered architectures, forward processing, and continuous error optimization.

- **Core Architecture**
  - Perceptron & hidden layers
  - Activation functions (e.g., ReLU, Sigmoid)
- **Forward Processing & Evaluation**
  - Feedforward feature learning
  - Error function measurement (MSE, Cross-entropy)
- **Optimization & Learning**
  - Backpropagation of errors
  - Gradient descent algorithms (Batch, SGD)

Traditional machine learning flow

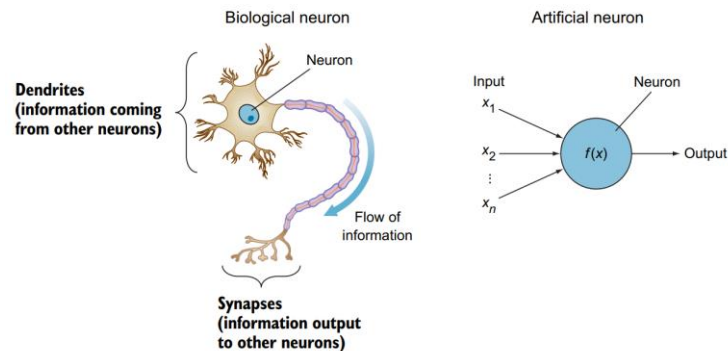
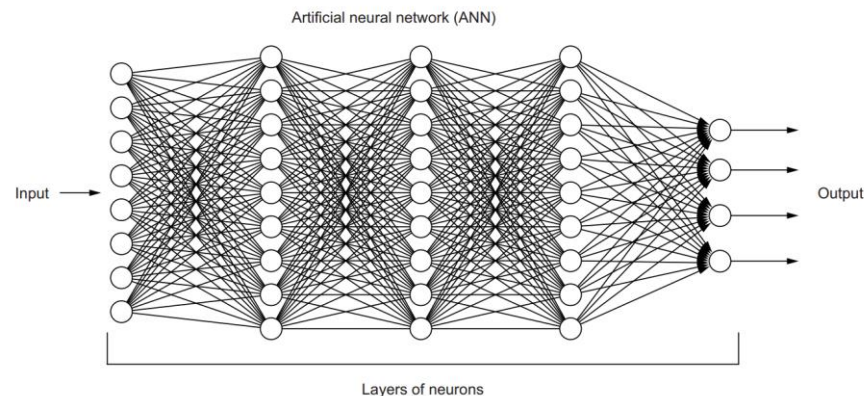


Deep learning flow



The perceptron is a single artificial neuron that mimics biological decision-making using weighted inputs and thresholds.

- **Network Foundation**
  - Simplest neural network (single neuron)
  - Building block for Multi-Layer Perceptrons (MLPs)
- **Processing Mechanism**
  - Calculates the weighted sum of inputs
  - Applies a step function (fires 0 or 1 based on threshold)
- **Connection Weights**
  - Determine the importance of each input feature
  - High weights amplify signals; low weights diminish them



A perceptron processes input data through a weighted sum and an adjustable bias to generate predictions.

- Inputs & Weights**

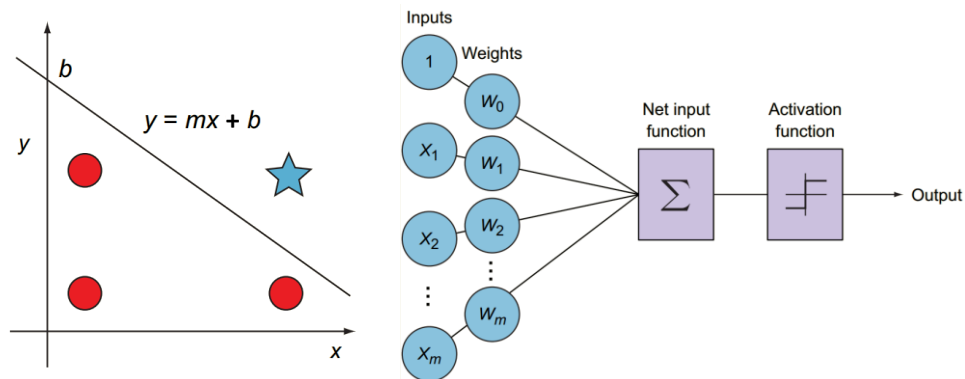
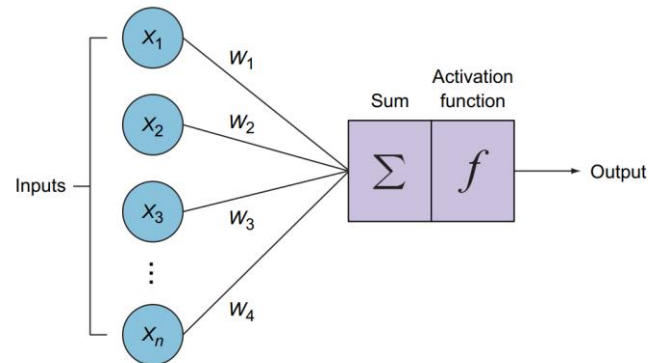
- Input vector ( $X$ ) represents data features.
- Weights ( $W$ ) define the importance of each feature.

- Weighted Sum Function** `z = np.dot(w.T, X) + b`

- Calculates the linear combination:  $z = X.W + b$ .
- Result feeds directly into the activation function.

- The Role of Bias ( $b$ )**

- Acts as the y-intercept for the function.
- Shifts the decision boundary to improve data fitting.



# Perceptron Activation and Learning

Perceptrons make binary decisions via step functions and learn by iteratively minimizing prediction errors.

- **Step Activation Function**

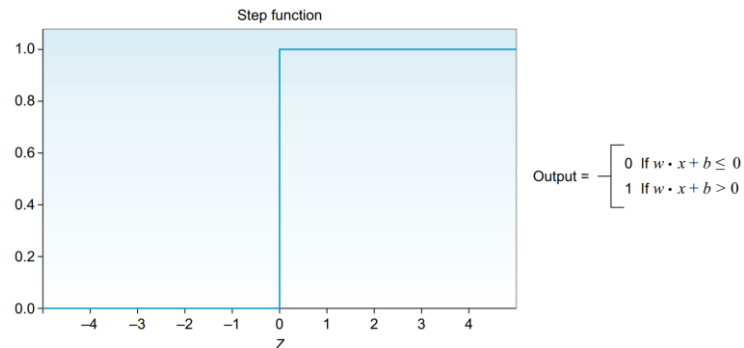
- Evaluates weighted sum:  $z = \sum x_i \cdot w_i + b$
- Outputs 1 (fires) if  $z \geq 0$ , else outputs 0

- **Error Measurement**

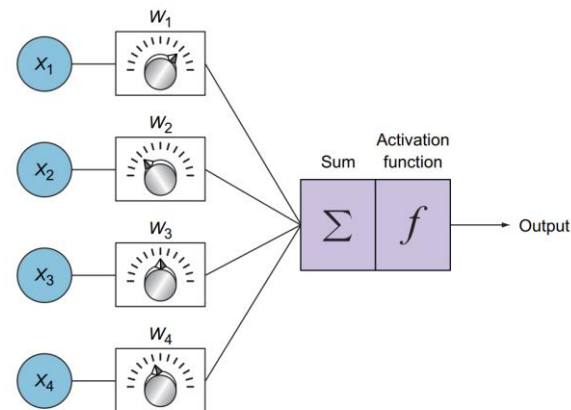
- Executes feedforward prediction ( $\hat{y}$ )
- Calculates prediction error:  $y - \hat{y}$

- **Iterative Learning Process**

- Adjusts weight values to correct predictions
- Repeats cycle until error approaches zero

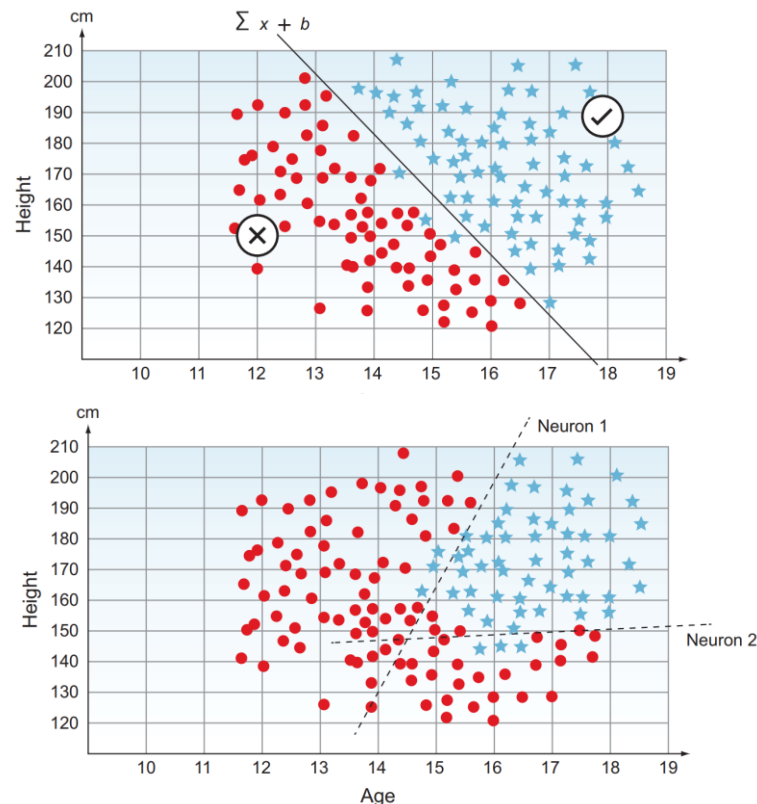


$\hat{y} = g(z)$ , where  $g$  is an activation function and  $z$  is the weighted sum  $= \sum x_i \cdot w_i + b$



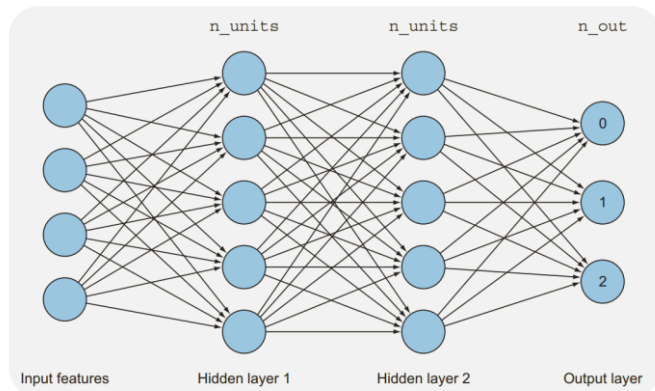
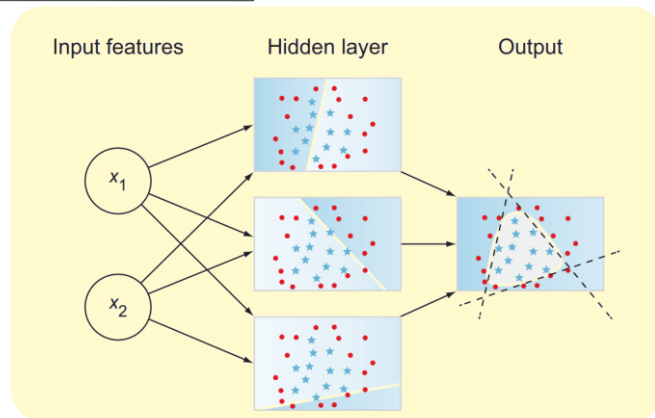
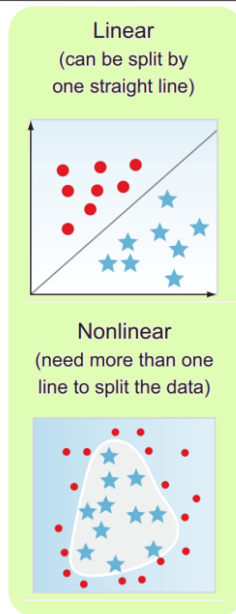
Single perceptrons are limited to linear decision boundaries, requiring multi-neuron networks to solve complex, non-linear problems.

- **Linear Limitations**
  - Produces a single straight-line boundary
  - Fails on non-linearly separable data
- **Multi-Neuron Solutions**
  - Combines neurons for complex boundaries
  - Successfully separates non-linear datasets
- **Balancing Complexity**
  - More neurons improve feature learning
  - Excessive neurons risk data overfitting



Stacking fully connected neuron layers empowers networks to learn complex, non-linear data patterns.

- **Tackling Non-Linearity**
    - Combines multiple neurons for complex boundaries
    - Overcomes single-line perceptron limitations
  - **Core Architectural Structure**
    - Comprises input, hidden, and output layers
    - Hidden layers progressively extract complex features
  - **Balancing Network Depth**
    - Deeper networks improve training data fit
    - Excessive layer complexity risks overfitting
- <https://playground.tensorflow.org/>





Neural networks learn complex patterns by iteratively optimizing weights across layered architectures, guided by strategically tuned hyperparameters.

## Core Architecture & Learning

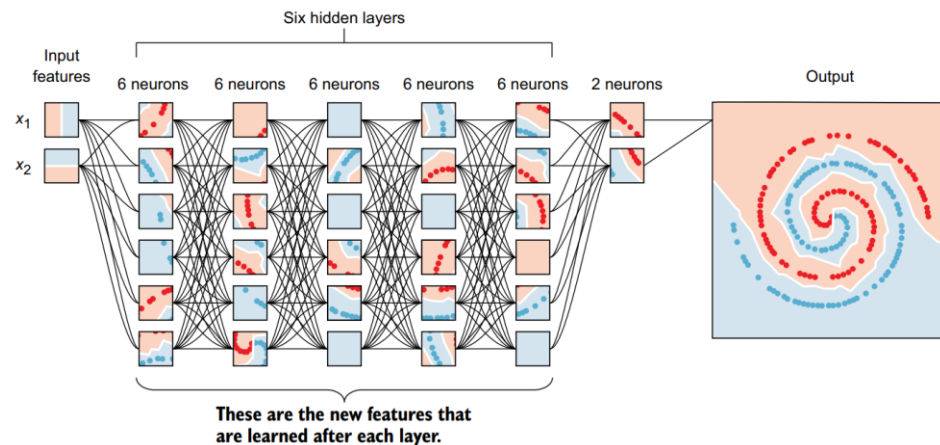
- **Structure:** MLPs use inputs, hidden layers, and outputs for non-linear data.
- **Process:** Feedforward (prediction) → Calculate Error → Backpropagate (weight update).

## Essential Hyperparameters

- **Layers & Nodes:** Balance depth to avoid overfitting (memorizing) or underfitting.
- **Functions:** Use ReLU (hidden) and Softmax (output); MSE (regression) or Cross-entropy (classification).

## Optimization & Tuning

- **Training Settings:** Tune Batch Size, Epochs, and Optimizers (e.g., SGD).
- **Learning Rate:** Crucial for controlling the speed and success of error minimization.



# Install & Use Pytorch

## 1 Why PyTorch?

Imagine a robotic arm picking delicate glass bottles on a conveyor. A vision system must be precise.



**TensorFlow (older style)**

Build the whole network like a "black box".

Hard to debug when things go wrong.

**PyTorch**

Dynamic & transparent. Inspect any step, see values inside the network.

Feels like native Python.

★ PyTorch lets you debug easily and see exactly what your model is doing at every step!

## 2 Key Concepts: Language, Framework & IDE

Let's understand with a cooking (soup) analogy!

**Programming Language**

Ingredients



Raw materials & rules (e.g., Python). You use them to create something.

**Framework**

Ready-made soup mix



Someone built it with the ingredients. You just use it (e.g., PyTorch).

**IDE (Environment)**

Pot & Stove



The tool/space where you cook and write code (e.g., PyCharm).

💡 PyTorch = Framework for Deep Learning  
PyCharm = IDE to write and manage code

## 3 Install PyTorch (Using pip)

Go to [pytorch.org](https://pytorch.org) and select your configuration.

|               |                |                                                      |
|---------------|----------------|------------------------------------------------------|
| PyTorch Build | Stable (1.4) ✓ | Preview (1.5+)                                       |
| Your OS       | Windows ✓      | macOS Linux                                          |
| Package       | Conda pip ✓    | LibTorch Source                                      |
| Language      | Python ✓       | (Automatically selected when pip or conda is chosen) |
| CUDA          | None (CPU) ✓   | 9.2 10.1 10.2 11.x                                   |

- Copy the command from "Run this Command" and run it.

```
pip install torch torchvision torchaudio
```

- Open CMD as Administrator, paste the command and install.

```
C:\> pip install torch torchvision torchaudio
Collecting torch ... Done
Collecting torchvision ... Done
Collecting torchaudio ... Done
Installing collected packages: torch, torchvision, torchaudio
Successfully installed torch-1.4.0 torchvision-0.5.0 torchaudio-0.4.0
```

- Test the installation

```
C:\> python
>>> import torch
>>> x = torch.rand(3, 3)
>>> print(x)
```

tensor([[[0.7823, 0.5462, 0.3661],
 [0.1439, 0.9748, 0.2639],
 [0.7186, 0.1753, 0.6869]])])

✓ Success!

⚠ Tip: If you get "Permission denied", run CMD as Administrator.

## 4 Install PyCharm (IDE)

Go to [jetbrains.com/pycharm](https://jetbrains.com/pycharm)

- Click Download
- Choose Community (free)
- Install like any other software (Next > Next > Finish)
- Available for Windows, macOS, Linux

💡 Note: Do not confuse **PyTorch** (framework) with **PyCharm** (IDE)!

## 5 Google Colab (The Easy Way!)

Free cloud notebooks with CPU/GPU. No installation needed!

- Sign in with Google
- File > New notebook
- You get a new notebook!

**Key Points**

- Click Connect → Connected RAM: 12.7 GB | Disk: 107 GB
- Write code in cells (gray boxes) `print("Hello, PyTorch!")`
- Click ▶ to run the cell
- Add new cell here + Code + Text
- Use GPU: Runtime > Change runtime type → Hardware accelerator GPU

♥ If you don't need GPU, keep it off so others can use it too!

**Common Mistakes**

- Wrong CUDA version for GPU
- Using old PyTorch version
- Not running CMD as Administrator

**What's Next?**

All set! In the next session we'll write our first PyTorch code.

**Remember**

Ingredients (Language) + Ready Mix (Framework) + Tools (IDE) = Powerful recipe for building intelligent systems!

# Install & Use Pytorch (Offline)

## 1 Install Python (from local sources)



Download Python from local sites (e.g., [Soft98](#))



Choose the right version for your system (e.g., Windows 64-bit)

**CRUCIAL: Check "Add Python to PATH"**

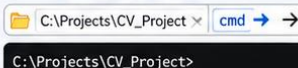
☒ Add Python to PATH

Avoids "python is not recognized" errors in terminal!

### PRO TIP: Open Terminal Here



Go to your project folder. Click the address bar, delete text, type `cmd` and press Enter. Terminal opens in your project path!



## 2 Create Virtual Environment (Best Practice)



Pro engineers isolate each project to avoid package conflicts.

### 1 Open terminal in your project folder

```
C:\Projects\CV_Project> python -m venv cv_env
```

A folder named `cv_env` will be created.



### 2 Activate the environment

```
C:\Projects\CV_Project> cv_env\Scripts\activate
```

When activated, `(cv_env)` appears at the beginning of the prompt.



```
(cv_env) C:\Projects\CV_Project>
```

☒ Everything you install now stays only in this project. You are fully isolated!

## 3 Bypass Sanctions with Local Mirrors (The Core!)



Official PyPI is blocked. Use local mirrors (copies) available in the country.



Recommended mirror:  
<https://iran.chabokan.net>  
We tell pip to install from this server.

### Install required libraries:

```
pip install --index-url https://mirror2.chabokan.net/pypi/simple/ opencv-python numpy matplotlib
```

☒ Fast, reliable, no need for VPN!



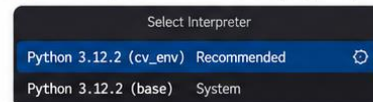
For deep learning (e.g., PyTorch): Install CPU version for now. GPU version needs heavy NVIDIA drivers—difficult in these conditions.

## 4 Select Environment in VS Code



VS Code may use the system Python instead of your virtual environment.

- Press **Ctrl** + **Shift** + **P**
- Type: **Python: Select Interpreter**
- Select your environment (`cv_env`)



### Test it!



```
import cv2
import numpy as np
print(cv2.__version__)
```



If it prints the OpenCV version, everything is set up successfully!



**Congratulations!** You're ready to code without limits!



Hope these unprecedented outages end soon and we return to stable internet. Our focus should be on learning, not bypassing restrictions—but until then, **we don't stop!**



No Internet?



Find a Solution



Keep Learning



Never Stop



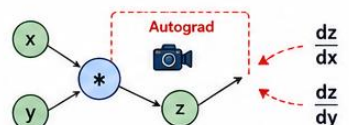
## 1 Foundations & Raw Materials (Tensors & GPU)



- Tensors: containers for our robot's camera pixels.
- Move data from CPU to GPU for massive speed.
- Set random seed for reproducible and isolated environments.

```
torch.manual_seed(42)
x = torch.randn(32, 3, 224, 224).to('cuda')
```

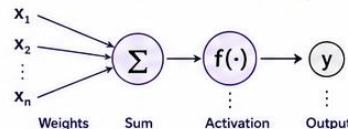
## 2 The Magic Engine of Derivatives (Autograd)



- Auto differentiation is PyTorch's heart.
- Records operations like a hidden camera.
- Computes billions of derivatives in a fraction of a second.

```
x.requires_grad_(True)
z = (x * y + y).sum(); z.backward()
```

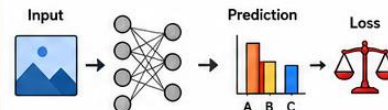
## 3 Brain Architecture & Activations (Architecture & Activations)



- Start with a simple neuron (perceptron).
- See its limits (fails on XOR).
- Stack layers → Deep Network (MLP).
- Activations: ReLU, Softmax enable learning complex patterns.

```
nn.Linear → ReLU → Linear → Softmax
```

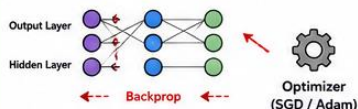
## 4 Test, Guess & Judge (Forward Pass & Loss Functions)



- Data flows forward (feedforward).
- The network makes a guess.
- Loss functions (MSE, Cross-Entropy) measure the disaster!

```
y_hat = model(x)
loss = criterion(y_hat, y_true)
```

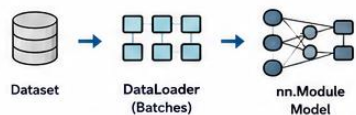
## 5 Learn & Correct the Path (Backpropagation & Optimizers)



- Error signal flows backward.
- Find out how much each weight is guilty!
- Optimizers adjust weights to reduce the next error.

```
loss.backward()
optimizer.step()
```

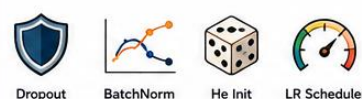
## 6 Entering the Engineering World (DataLoaders & PyTorch Modules)



- Stop manual matrix handling.
- Use nn.Module for clean, reusable models.
- DataLoader feeds huge data in batches fast and organized.

```
dataset → DataLoader → model(batch)
```

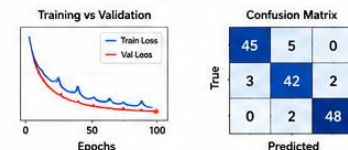
## 7 Real-World Challenges & Defenses (Regularization)



- Prevent overfitting (memorizing instead of learning).
- Use defenses: Dropout, BatchNorm.
- Initialize smartly (He Initialization).
- Control learning speed with LR Schedulers.

```
scheduler.step()
```

## 8 Diagnostics & Deployment (Diagnostics & Deployment)



- Diagnose with learning curves & confusion matrix.
- Save the trained brain and deploy it on real robots.

```
torch.save(model.state_dict(), 'model.pth')
```